

Q5.1) a) Explain the role of pooling layer in Convolution neural network

In a Convolutional Neural Network (CNN), the pooling layer plays a crucial role in reducing the spatial dimensions of the input feature maps, leading to a more compact representation while preserving the most important features. The pooling operation is applied independently to each depth slice of the input feature maps, also known as point-wise, and involves aggregating information over local regions. Here's a point-wise explanation of the role of the pooling layer:

1. **Spatial Down-sampling:** The pooling layer reduces the spatial dimensions (width and height) of each feature map independently. This is achieved by dividing the input feature map into non-overlapping regions (e.g., 2x2 or 3x3 windows) and applying a pooling operation (e.g., max pooling or average pooling) to each region to obtain a single output value.
2. **Translation Invariance:** Pooling helps create translation invariance in the network by aggregating information from local regions. This means that the network can still recognize patterns even if they are shifted slightly in the input. By aggregating information point-wise, the pooling layer helps capture the presence of features regardless of their precise location in the input.
3. **Dimensionality Reduction:** By reducing the spatial dimensions of each feature map independently, the pooling layer reduces the computational complexity of the network. This is particularly important in deep networks with many layers, as it helps control the number of parameters and computations required.
4. **Feature Retention:** While reducing the spatial dimensions, the pooling layer retains the most important features from each region. In max pooling, for example, only the maximum value in each region is retained, which tends to preserve the most salient features in the input.
5. **Downstream Processing:** The output of the pooling layer serves as input to the next layer (e.g., a convolutional layer or a fully connected layer). By reducing the spatial dimensions, the pooling layer helps reduce the number of parameters and computations in the downstream layers, making the network more efficient.

5.2) Explain the concept of transfer learning and its importance in deep learning

Transfer learning is a machine learning technique where a model trained on one task is re-purposed on a second related task. It is a popular approach in deep learning, especially when working with limited labeled data or computational resources. Here's a breakdown of the concept and its importance:

1. **Concept:** In transfer learning, a pre-trained model (often trained on a large dataset) is used as the starting point for a new task, instead of training a new model from scratch. The pre-trained model has already learned general features from the original task, which can be useful for the new task. The final layers of the pre-trained model are often replaced or retrained to adapt to the new task.
2. **Importance:**
 - **Reduced Training Time:** Transfer learning can significantly reduce the time and resources required to train a new model, as the pre-trained model has already learned many relevant features.
 - **Improved Performance:** Transfer learning can lead to better performance on the new task, especially when the new task has limited training data. The pre-trained model provides a good starting point, and fine-tuning the model on the new task can lead to better generalization.
 - **Robustness:** Transfer learning can help improve the robustness of a model, as the features learned from the original task may capture important patterns that are useful across different tasks.
 - **Domain Adaptation:** Transfer learning is also useful for domain adaptation, where the distribution of the data in the new task is different from the original task. The pre-trained model can provide a good initialization for the new task, which can then be further adapted to the new domain.

Overall, transfer learning is a powerful technique in deep learning that leverages pre-trained models to improve performance, reduce training time, and enhance the robustness of models across a wide range of tasks and domains.

Q.5 .3) Explain Padding in neural network.

Padding in neural networks refers to the process of adding extra layers of zeros around the input data, typically in convolutional neural networks (CNNs). Padding is used to control the spatial dimensions of the output volume, especially when applying convolutional operations.

There are two main types of padding:

1. **Valid Padding (No Padding):** In valid padding, no padding is added to the input data. As a result, the spatial dimensions of the output volume decrease compared to the input volume, especially after applying convolutional operations with a filter/kernel.
2. **Same Padding:** In same padding, zeros are added to the input data so that the spatial dimensions of the output volume remain the same as the input volume. This is achieved by adding zeros around the input data such that the filter/kernel can be centered on each element of the input.

The choice of padding affects the spatial dimensions of the output volume after applying convolutional operations. Valid padding typically leads to a reduction in spatial dimensions, which can result in loss of information at the edges of the input. On the other hand, same padding helps preserve the spatial dimensions and can help prevent information loss, especially at the edges.

Padding is often used in conjunction with convolutional layers to control the size of the output feature maps and ensure that the network learns useful spatial hierarchies of features. It is a common technique used in CNNs to improve the performance of the network, especially when dealing with images or other spatial data.

Q5.1) a) Explain Residual network in Convolution neural network.

A Residual Network, or ResNet, is a type of Convolutional Neural Network (CNN) architecture that was designed to address the problem of vanishing gradients in very deep neural networks. ResNets introduced the concept of "skip connections" or "shortcut connections," which allow gradients to flow more easily during training, enabling the training of very deep networks (hundreds or even thousands of layers) without performance degradation. **Here's how Residual Networks work:**

1. **Basic Building Block:** The basic building block of a ResNet is called a "residual block." A residual block consists of two main components: a "shortcut connection" and a series of convolutional layers.
2. **Shortcut Connection:** The shortcut connection is a direct path that skips one or more layers in the network. It allows the input to be directly added to the output of the residual block. This concept is illustrated below:

INPUT --→ CONVOLUTION---→ ACTIVATION---→CONOLUTION--
→OUTPUT

3.Identity Mapping: The main idea behind ResNets is the use of identity mapping, where the network is encouraged to learn a residual mapping (the difference between the input and output of a block) instead of directly learning the desired output mapping. This makes it easier for the network to learn the identity mapping (i.e., do nothing) if it is the optimal mapping for a given layer. **4. Residual Learning:** The output of a residual block is computed as the sum of the input and the residual mapping learned by the block. Mathematically, this can be represented as: **Output=Input+Residual**

5 .Benefits:

Addressing Vanishing Gradients: By using shortcut connections, ResNets allow gradients to flow directly through the network, mitigating the vanishing gradient problem that can occur in very deep networks.

Easier Optimization: The use of residual connections makes it easier to train very deep networks, as the network can learn to make small adjustments to the identity mapping rather than learning complex transformations from scratch.

6.Architecture: In practice, ResNets are built by stacking multiple residual blocks together. The full ResNet architecture typically consists of several stages, each containing multiple residual blocks, followed by a global average pooling layer and a fully connected layer for classification

5.2) Explain the concept of SoftMax regression and its significance in CNN models.

Softmax regression, also known as multinomial logistic regression, is a type of regression model used for classification tasks where the output is a set of mutually exclusive classes. Softmax regression is particularly useful in Convolutional Neural Networks (CNNs) for the final classification layer, where it converts the raw output scores from the previous layers into class probabilities. **Working : Output Layer:** In a CNN, the output layer is typically a softmax layer. Each neuron in the softmax layer represents a class, and the output of the neuron is the probability that the input image belongs to that class.

1. **Softmax Function:** The softmax function is used to convert the raw output scores (logits) from the previous layer into probabilities. The softmax function is defined as:

$$P(y_i|\mathbf{x}) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Where:

- $P(y_i|\mathbf{x})$ is the probability that the input $\mathbf{x}$ belongs to class i ,
 - z_i is the raw score (logit) for class i ,
 - The sum is taken over all classes.
2. **Normalization:** The softmax function normalizes the raw scores into a valid probability distribution, where the sum of all class probabilities is equal to 1. This allows the model to output meaningful probabilities that can be interpreted as the confidence of the model in its predictions.
 3. **Cross-Entropy Loss:** Softmax regression is typically trained using the cross-entropy loss function, which measures the difference between the predicted class probabilities and the true class labels. The goal of training is to minimize the cross-entropy loss, which encourages the model to output high probabilities for the correct class and low probabilities for incorrect classes.
 4. **Significance in CNNs:** Softmax regression is crucial in CNNs for converting the output of the convolutional and pooling layers into class probabilities. By using the softmax function, CNNs can make probabilistic predictions and provide a measure of uncertainty in their predictions. This is especially important in tasks like image

Q5.4) What frameworks are used in deep learning? Define any seven.

There are several popular frameworks used in deep learning, each with its strengths and areas of focus. Here are seven of them:

1) TensorFlow: Developed by Google Brain, TensorFlow is one of the most widely used deep learning frameworks. It offers a flexible architecture for numerical computation across multiple platforms and devices.

TensorFlow provides high-level APIs for building and training neural networks, as well as lower-level operations for more advanced

customization. **2) PyTorch:** Developed by Facebook's AI Research lab (FAIR), PyTorch has gained significant popularity due to its dynamic computational graph feature and intuitive interface. It allows for easy experimentation with neural network architectures and supports dynamic computation, making it favored by researchers and practitioners alike

3) Keras: Keras is an open-source neural network library written in Python. It is known for its user-friendly API and ease of use, making it a popular choice for beginners and rapid prototyping. Keras can run on top of TensorFlow, Theano, or Microsoft Cognitive Toolkit (CNTK), providing a high-level interface for building and training neural networks.

4) Caffe: Caffe is a deep learning framework developed by the Berkeley Vision and Learning Center (BVLC). It is known for its expressive architecture and efficiency, particularly for convolutional neural networks (CNNs)

5) MXNet: MXNet is an open-source deep learning framework developed by Apache Software Foundation. It offers a scalable and efficient implementation of neural networks, with support for both imperative and symbolic programming. MXNet provides APIs in multiple programming languages, including Python, R, Julia, Scala, and C++.

6) TensorFlow.js: TensorFlow.js brings the capabilities of TensorFlow to JavaScript environments, allowing developers to build and train machine learning models directly in the browser or on Node.js

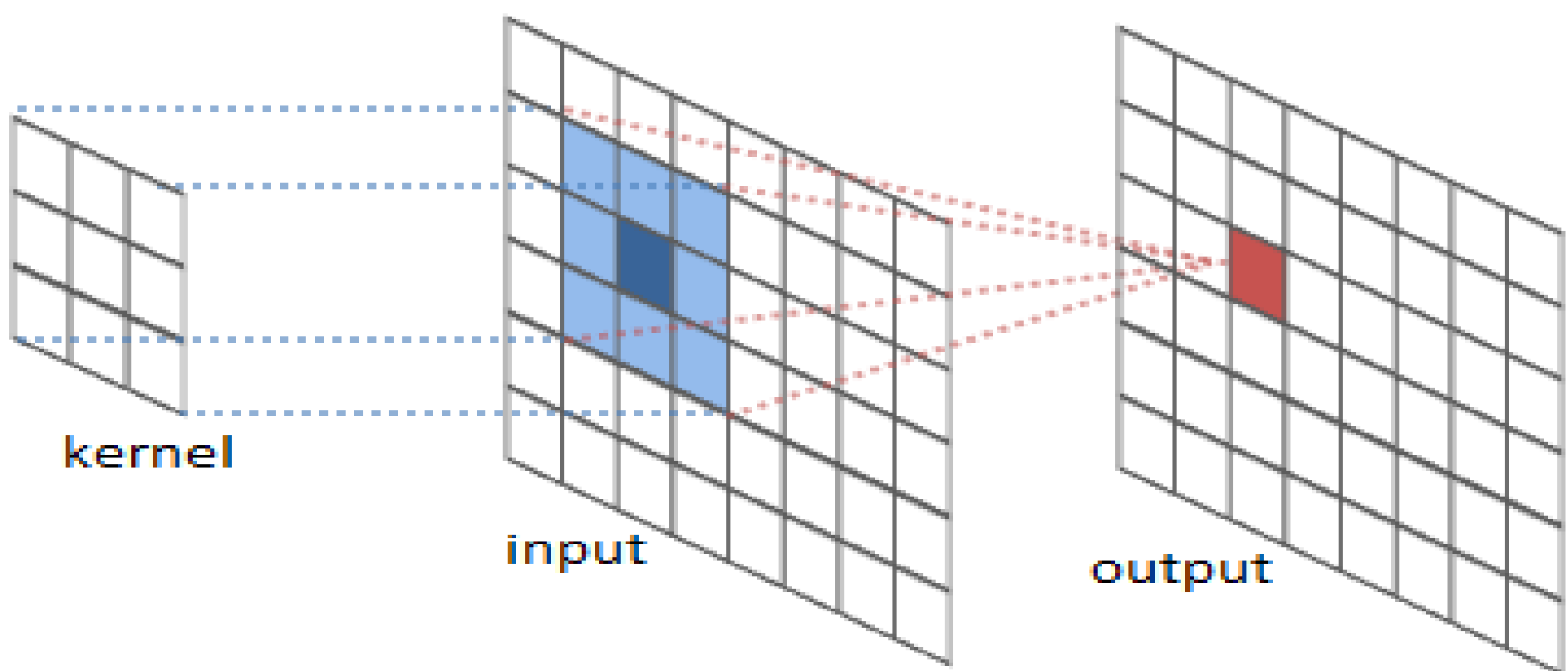
7) CNTK (Microsoft Cognitive Toolkit): CNTK is a deep learning framework developed by Microsoft Research. It is known for its efficiency and scalability, particularly in distributed training scenarios. CNTK provides a toolkit for building and training deep neural networks with support for multiple programming languages, including Python, C++, and C#.

Q5.5) Exemplify convolution over volume with convolution on RGB images. Also illustrate multiple filters used in it.

Suppose, instead of a 2-D image, we have a 3-D input image of shape $6 * 6 * 3$. How will we apply convolution on this image? We will use a $3 * 3 * 3$ filter instead of a $3 * 3$ filter. Let's look at an example:

Input: $6 * 6 * 3$ Filter: $3 * 3 * 3$

- The dimensions above represent the height, width and channels in the input and filter. Keep in mind that the number of channels in the input and filter should be same. This will result in an output of $4 * 4$



1

Since there are three channels in the input, the filter will consequently also have three channels. After convolution, the output shape is a $4 * 4$ matrix. So, the first element of the output is the sum of the element-wise product of the first 27 values from the input (9 values from each channel) and the 27 values from the filter. After that we convolve over the entire image.

Instead of using just a single filter, we can use multiple filters as well. How do we do that? Let's say the first filter will detect vertical edges and the second filter will detect horizontal edges from the image. If we use multiple filters, the output dimension will change. So, instead of having a $4 * 4$ output as in the above example, we would have a $4 * 4 * 2$ output (if we have used 2 filters):

Q5.6) Consider a LeNet-5 a convolutional neural network, we want to perform the classification of digits, Write down the complete procedure followed in its architecture

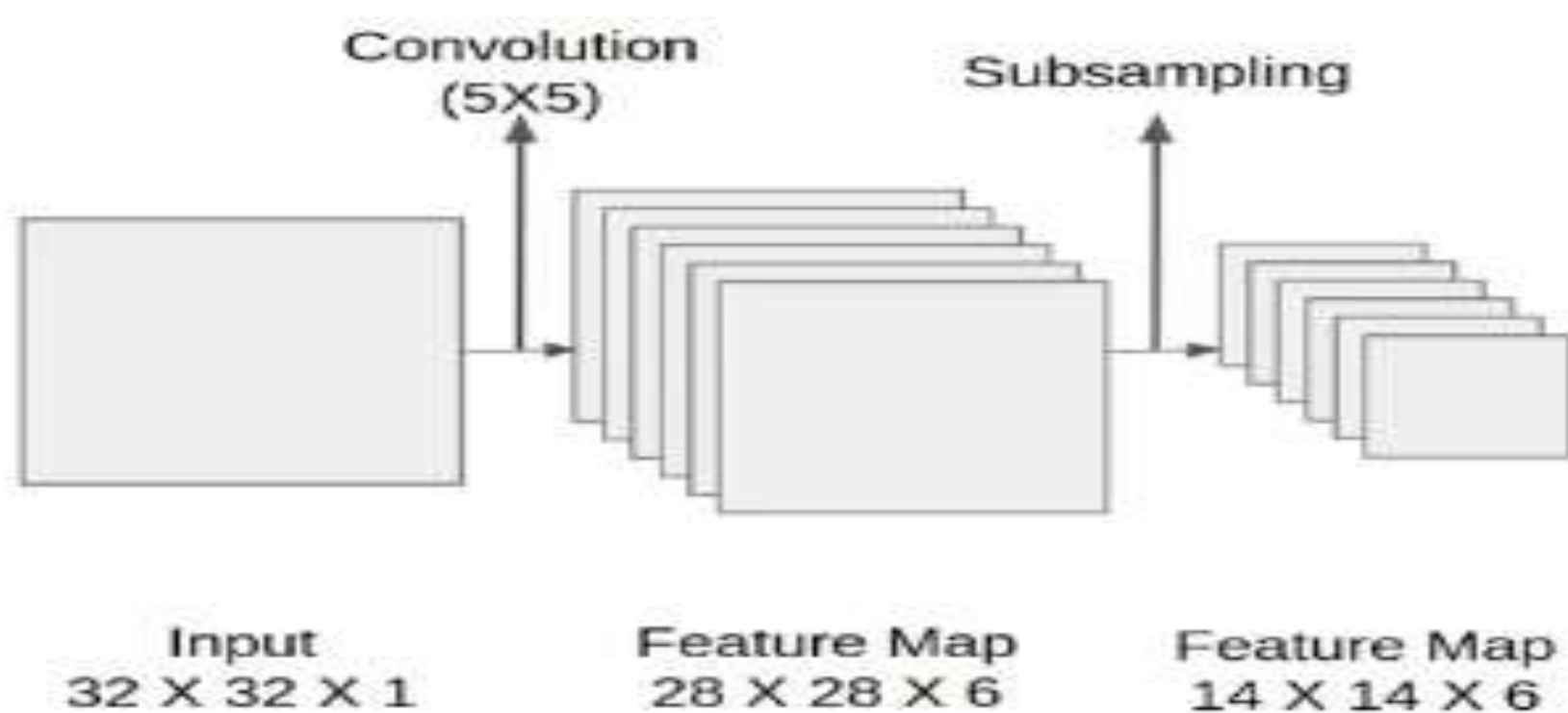
LeNet-5 is a convolutional neural network architecture that was created by Yann LeCunn in 1998. It includes 7 layers, excluding the input layer, which contains the trainable parameters called weights. LeNet-5 consists of two parts: i) A convolutional encoder consisting of two convolutional layers, and ii) A dense block consisting of three fully connected layers. Layer C1 is a convolutional layer with six feature maps where the size of the feature maps is 28×28 ;

- Layer S2 is a sub-sampling layer with six feature maps where the size of the feature maps is 14×14

Layer C3 is a convolutional layer with sixteen feature maps where the size of the feature maps is 10×10 :

Layer S4 is a sub-sampling layer with sixteen feature maps where the size of feature maps is 5×5

Layer C5 is a convolutional layer with 120 feature maps where the size of the feature maps is 1×1



- The ability to process higher resolution images requires larger and more convolutional layers, so this technique is constrained by the availability of computing resources.

Q5.7) What is transfer learning models for image classification? What are the 5 types of transfer learning?

Transfer learning generally refers to a process where a model trained on one problem is used in some way on a second related problem.

- In deep learning, transfer learning is a technique whereby a neural network model is first trained on a problem similar to the problem that is being solved. Transfer learning has the advantage of decreasing the training time for a learning model and can result in lower generalization error. Transfer learning is about leveraging feature representations from a pre-trained model, so we don't have to train a new model from scratch. The pre-trained models are usually trained on massive datasets that are a standard benchmark in the computer vision frontier. The weights obtained from the models can be reused in other computer vision tasks.

Steps for transfer learning models for image classification:

Obtain pre-trained model: The first step is to choose the pre-trained model we would like to keep as the base of our training, depending on the task..

Create a base model: The base model is one of the architectures such as ResNet which we have selected in the first step to be in close relation to our task. We can either download the network weights which saves the time of additional training of the model.

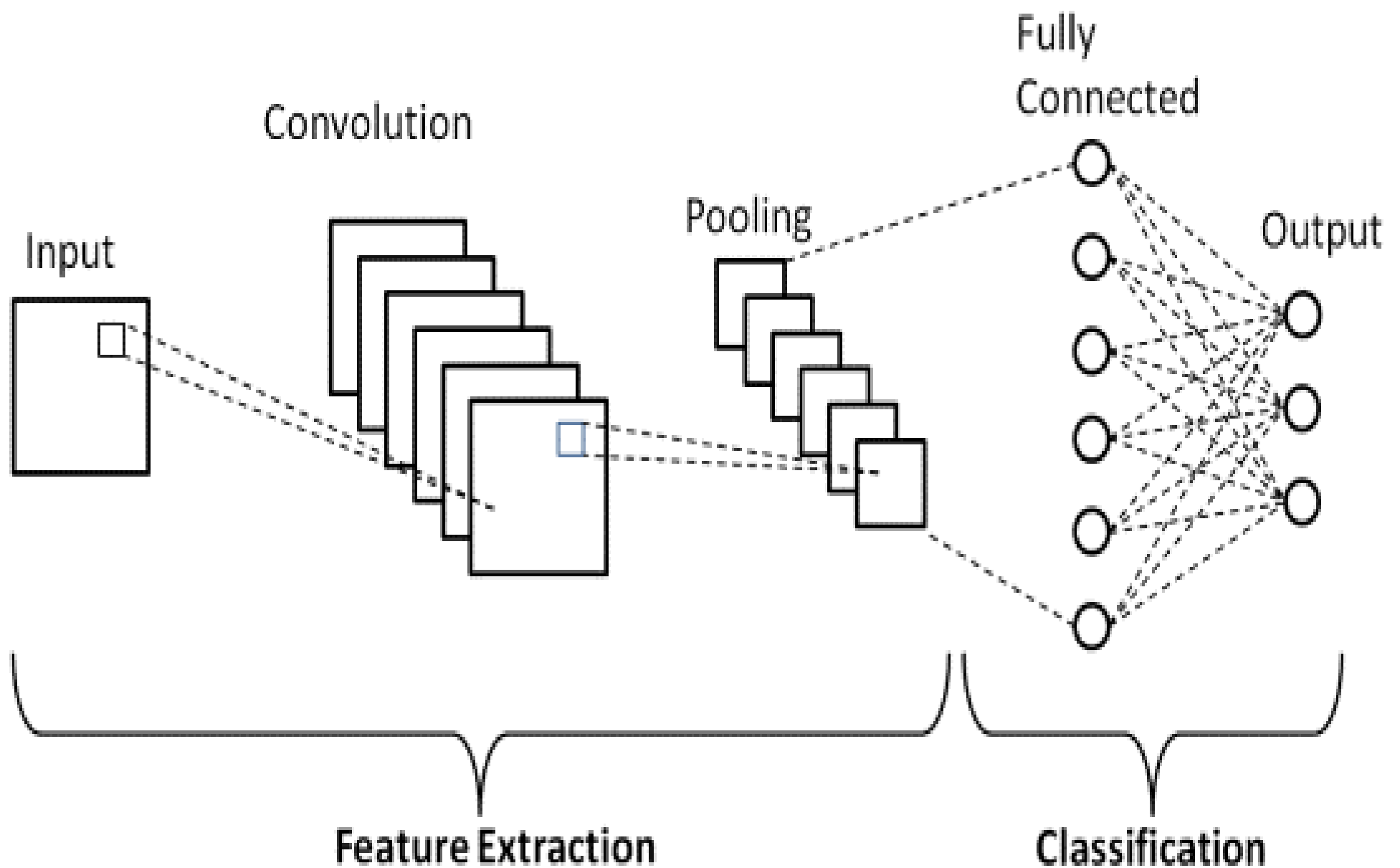
Freeze layers: Freezing the starting layers from the pre-trained model is essential to avoid the additional work of making the model learn the basic features. If we do not freeze the initial layers, we will lose all the learning that has already taken place

Add new trainable layers: The only knowledge we are reusing from the base model is the feature extraction layers. We need to add additional layers on top of them to predict the specialized tasks of the model. These are generally the final output layers.

Train the new layers: The pre-trained model's final output will most likely differ from the output we want for our model. For example, pre-trained models trained on the ImageNet dataset will output 1000 classes.

Q5.8) Explain architecture of Convolution Neural Networks (ConvNet).

Ans. The architecture of a CNN is designed to take advantage of the 2D structure of an input image.



CNN consists of a number of convolutional and sub-sampling layers optionally followed by fully connected layers. The input to a convolutional layer is "mxmxr" image where m is the height and width of the image and r is the number of channels. CNN architecture is made up of several layers that implement feature extraction, and then classification.

The image is divided into receptive fields that feed into a convolutional layer, which then extracts features from the input image. The next step is pooling, which reduces the dimensionality of the extracted features (through down-sampling) while retaining the most important information.

Another convolution & pooling step performed feeds into a fully connected multilayer perceptron. The final output layer network is a set of nodes that identify features of image. You train network by using back-propagation.

An input image is passed to the first convolutional layer. The convoluted output is obtained as an activation map. The filters applied in convolution layer extract relevant features from input image to pass further.

- Each filter shall give a different feature to aid the correct class prediction. In case we need to retain the size of the image, we use same padding (zero padding), otherwise valid padding is used since it helps to reduce the number of features.
- Pooling layers are then added to further reduce the number of parameters. Several convolution and pooling layers are added before the prediction is made. Convolutional layer help in extracting features..

The output layer in a CNN as mentioned previously is a fully connected layer, where the input from the other layers is flattened and sent so as to transform the output into the number of classes as desired by the network.

- The output is then generated through the output layer and is compared to the output layer for error generation.

- A loss function is defined in the fully connected output layer to compute the mean square loss. The gradient of error is then calculated.

The error is then backpropagated to update filter (weights) & bias values. One training cycle is completed in a single forward & backward pass.

Max pooling : Max pooling is a rule to take the maximum of a region and help to proceed with the most crucial features from the image. It is a sample-based process that transfers continuous functions into discrete counterparts. Its primary objective is to downscale an input by reducing its dimensionality and making assumptions about features contained in the sub-region that were rejected..

Q5.3) Illustrate with example convolution and max pooling?

Convolution: Convolution is a fundamental operation in deep learning for processing structured data, especially in computer vision tasks. In convolutional neural networks (CNNs), convolutional layers learn features from input data by applying a set of filters (also called kernels) across the input image. These filters slide over the input image, performing element-wise multiplication with the overlapping region and then summing up the

results. This process helps in capturing local patterns and features from the input. Imagine you have a grayscale image of a 5x5 pixel grid. A 3x3 filter slides over this image, computing the dot product of the filter values and

Q5.9) Write short note on: AlexNet.

Ans. Alexnet is a Deep Convolutional Neural Network for image classification.

- AlexNet is composed of 5 convolutional layers with a combination of max-pooling layers, 3 fully connected layers, and 2 dropout layers. The activation function used in all layers is Relu. The activation function used in the output layer is Softmax. The total number of parameters in this architecture is around 60 million.

The AlexNet architecture was created with large-scale image datasets in mind, and it produced state-of-the-art results when it was first released. It has 60 million characteristics in all. For the AlexNet architecture, the convolutional kernels are extracted during the back-propagation optimization procedure by optimizing the whole cost function with the stochastic gradient descent algorithm.

Generally, the convolutional layers act upon the input feature maps with the sliding convolutional kernels to generate the convolved feature maps, and the pooling layers operate on the convolved feature maps to aggregate the information within the given neighborhood window with a max pooling operation or average pooling operation.

- Max pooling is applied between every two conv layers. After the tensors are flattened, two fully-connected (dense) layers are used. The output layer is a softmax layer to compute the softmax loss function for learning
- AlexNet uses Rectified Linear Units (ReLU) instead of the tanh function, which was standard at the time. ReLU's advantage is in training time; a CNN using ReLU was able to reach a 25% error on the CIFAR-10 dataset six times faster than a CNN using tanh.
- Use of multiple GPUs and parallel computing is highlighted in the training of AlexNet. The use of ReLU as the activation function for image classification by CNN. Multiple GPUs: GPUs were still rolling around with 3 gigabytes of memory. This was especially bad because the training set had

1.2 million images. AlexNet allows for multi-GPU training by putting half of the model's neurons on one GPU and the other half on another GPU

Q5.10) What are the difference between Keras and Tensorflow?

Keras	Tensorflow
Keras is a high-level API which is running on top of TensorFlow.	TensorFlow is a framework that offers both high and low-level APIs.
Keras is used for small datasets as it is slower.	TensorFlow is used for high-performance models and massive datasets that require execution fast.
It can be used for low-performance models.	It is used for high-performance models.
Working with beginner-friendly projects with small datasets.	Rendering heavy projects with ease.
Readable and concise architecture.	Comparatively complex architecture.
Community support is minimal but growing.	Being one of the first deep learning frameworks that were developed, it has an extensive community and a rich reserve of readable material.

Q.24 Explain briefly bias-variance trade off.

Ans. : In the experimental practice we observe an important phenomenon called the bias variance dilemma.

- In supervised learning, the class value assigned by the learning model built based on the training data may differ from the actual class value. This error in learning can be of two types, errors due to 'bias' and error due to 'variance'.
- Give two classes of hypothesis (e.g. linear models and k-NNs) to fit to some training data set, we observe that the more flexible hypothesis class has a low bias term but a higher variance term. If we have parametric family of hypothesis, then we can increase the flexibility of the hypothesis but we still observe the increase of variance.
- The bias-variance-dilemma is the problem of simultaneously minimizing two sources of error that prevent supervised learning algorithm from generalizing beyond their training set:
 1. The bias is error from erroneous assumptions in the learning algorithm. High bias can cause an algorithm to miss the relevant relations between features and target outputs.
 2. The variance is error from sensitivity to small fluctuations in the training set. High variance can cause overfitting: Modeling the random noise in the training data, rather than the intended outputs.
- In order to reduce the model error, the designer can aim at reducing either the bias or the variance, as the noise components is irreducible.
- As the model increases in complexity, its bias is likely to diminish. However, as the number of training examples is kept fixed, the parametric identification of the model may strongly vary from one DN to another, This will increase the variance term.

- At one stage, the decrease in bias will be inferior to the increase in variance, warning that the model should not be too complex. Conversely, to decrease the variance term, the designer has to simplify its model so that it is less sensitive to a specific training set. This simplification will lead to a higher bias.